LERN-ABHANDLUNG

Fussball Turnier Web

Eine Turnierverwaltung in HTML, CSS und Vanilla JavaScript – Aufbau, Denkweise und Technik zum Nachvollziehen und Nachbauen.

Stand 2026-10-09 · Grundlage: Projekt-Dokumentation „Fussball Turnier – die Web-Anwendung“ · Rw



Die Projekt-Dokumentation beschreibt, **was** die Anwendung tut und **wie** sie aufgebaut ist. Diese Abhandlung geht einen Schritt weiter: Sie erklärt, **warum** die Dinge so gebaut sind, welche allgemeinen Prinzipien dahinterstehen und wie man dieselben Ideen in eigenen Projekten wiederverwendet.

Jedes Kapitel folgt demselben Muster: erst das Prinzip, dann der echte Code aus dem Projekt, dann ein „Merke“-Kasten. Wo es hilft, steht der Vergleich zu Delphi daneben – denn viele Konzepte aus HTML/JS haben ein direktes Gegenstück in der VCL- oder TMS-Welt.

> **LERNZIELE**
>
> Nach dem Durcharbeiten kannst du …
>
> - erklären, welche Aufgabe HTML, CSS und JavaScript jeweils übernehmen und wie sie zusammenspielen,
> - eine Datenschicht als IIFE-Modul mit klarer öffentlicher Schnittstelle bauen,
> - Daten mit `localStorage` und JSON dauerhaft im Browser ablegen und robust wieder einlesen,
> - begründen, warum man berechenbare Werte (Punkte, Platzierungen) nicht speichert,
> - eine Ligatabelle mit `Map`, Spread-Operator und Vergleichsfunktion berechnen und sortieren,
> - typische Browser-Fallen vermeiden: `innerHTML`, `event.key`, Datums-Zeitzonen, das `storage`-Ereignis,
> - dieselbe Tabellenlogik in Delphi mit `TDictionary` und `TComparer` umsetzen.

Inhalt

Nr.	Kapitel	Teil
1	Drei Sprachen, drei Aufgaben	Grundlagen
2	Mehrseiten-App ohne Framework	Grundlagen
3	Das Modulmuster IIFE	Grundlagen
4	localStorage und JSON	Grundlagen
5	Abgeleitete Daten nicht speichern	Datenlogik
6	Die Tabelle Schritt für Schritt (mit Labor)	Datenlogik

7	Sortieren mit Vergleichsfunktion	Datenlogik
8	Defensive Programmierung	Datenlogik
9	IDs statt Namen	Datenlogik
10	DOM, Events und Sicherheit	Oberfläche
11	Seiten synchron halten	Oberfläche
12	Tastatur: `key` und `code` (mit Labor)	Oberfläche
13	CSS-Werkzeuge: Variablen, Grid, clamp, Medienabfragen, Druck	Oberfläche
14	Datum und Zeitzonen	Oberfläche
15	Sichern und Laden: Blob, FileReader, Promise	Oberfläche
16	Dieselbe Logik in Delphi	Vertiefen
17	Übungen mit Lösungen	Vertiefen
18	Glossar und Zusammenfassung	Vertiefen

TEIL 1

Grundlagen – wie eine Browser-Anwendung gebaut ist

1 Drei Sprachen, drei Aufgaben

Eine Web-Anwendung besteht aus drei Sprachen, die sich die Arbeit sauber teilen. Wer das einmal verinnerlicht hat, findet sich in jedem Web-Projekt zurecht.

Sprache	Aufgabe	Im Projekt	Gegenstück in Delphi
HTML	Struktur: *Was* steht auf der Seite?	7 Seiten, z. B. `tabelle.html`	Die `.dfm`-Datei: welche Komponenten es gibt und wie sie verschachtelt sind
CSS	Aussehen: *Wie* sieht es aus?	`global.css`, `layout.css`, je Seite eine Datei	Die Eigenschaften im Objektinspektor: `Color`, `Font`, `Align`, `Margins`
JavaScript	Verhalten: *Was passiert*, wenn man klickt?	`turnier-store.js` und je Seite ein Skript	Die `.pas`-Unit mit Ereignisbehandlern wie `Button1Click`

Der wichtigste Unterschied zu Delphi: Im Browser ist das Aussehen **nicht** an der Komponente gespeichert, sondern wird über **Selektoren** von außen zugewiesen. Eine Regel wie `.live-stand { font-size: … }` trifft jedes Element mit dieser Klasse – egal auf welcher Seite. Das entspricht eher einem VCL-Style, der für alle Formulare gilt, als den Einzeleigenschaften einer Komponente.

Ein kleines Beispiel aus dem Projekt zeigt, wie die drei Ebenen zusammenarbeiten. HTML markiert die Seite, CSS färbt den aktiven Menüpunkt, JavaScript verbindet beides:

`HTML – jede Seite sagt, wer sie ist`

```
<body data-page="spiele">
```

```
<body data-page="spiele">
  <nav class="main-nav">
    <a href="spiele.html" data-nav="spiele">Spiele</a> …
  </nav>
```

JavaScript – nav.js, die vollständige Datei

```
const page = document.body.dataset.page;
const link = document.querySelector(`.main-nav a[data-nav="${page}"]`);
if (link) link.classList.add("is-active");
```

CSS – layout.css

```
.main-nav a.is-active { background: var(--color-orange); color: #fff; }
```

JavaScript ändert hier keine Farbe direkt. Es setzt nur eine **Klasse**, und CSS entscheidet, wie diese Klasse aussieht. Diese Trennung hält beide Seiten austauschbar: Ein neues Farbschema berührt kein einziges Skript.

> **MERKE**
>
> HTML beschreibt, CSS gestaltet, JavaScript verändert Zustände – am besten nur über Klassen und Attribute, nicht über Inline-Stile.

---

2 Mehrseiten-App ohne Framework

Viele moderne Web-Anwendungen sind „Single-Page-Apps“: eine einzige HTML-Seite, deren Inhalt ein Framework wie React austauscht. Dieses Projekt geht bewusst den älteren, einfacheren Weg: **sieben echte HTML-Seiten**, zwischen denen ganz normale Links wechseln.

Vor- und Nachteile

Mehrseiten (dieses Projekt)	Single-Page-App mit Framework
Kein Build-Schritt; Ordner kopieren, `index.html` öffnen	Braucht npm, Bundler, Build
Jede Seite ist für sich verständlich	Zustand lebt im Speicher, Übergänge ohne Neuladen
Läuft auch in zehn Jahren noch	Werkzeugversionen veralten
Daten müssen zwischen Seiten gespeichert werden	Daten bleiben im Arbeitsspeicher

Der letzte Punkt ist der entscheidende: Wer von `spiele.html` zu `tabelle.html` wechselt, verliert alle JavaScript-Variablen, denn der Browser lädt eine neue Seite. Deshalb braucht eine Mehrseiten-App zwingend einen **gemeinsamen Speicher** – hier `localStorage` (Kapitel 4).

Reihenfolge der Skripte

Ohne Bundler gibt es kein `import`-Werkzeug, das Abhängigkeiten auflöst. Die Reihenfolge der `<script>`-Tags übernimmt diese Aufgabe:

```
<script src="js/turnier-store.js"></script>   <!-- 1. Datenschicht zuerst -->
<script src="js/nav.js"></script>             <!-- 2. Menü markieren -->
```

```
<script src="js/tabelle.js"></script>        <!-- 3. Seitenskript nutzt TurnierStore -->
```

Steht `tabelle.js` vor dem Store, gibt es beim Laden den Fehler `TurnierStore is not defined`. Das ist dasselbe Prinzip wie die `uses`-Klausel in Delphi – nur dass der Browser die Reihenfolge nicht selbst prüft.

> **DELPHI-VERGLEICH**
>
> Die sieben HTML-Seiten entsprechen sieben Formularen einer VCL-Anwendung. Der Unterschied: Delphi-Formulare leben gleichzeitig im selben Prozess und können sich direkt Daten zureichen. Browser-Seiten leben nacheinander und kennen sich nicht – sie teilen nur den Speicher.

---

3 Das Modulmuster IIFE

In klassischem JavaScript landet jede Funktion, die man auf oberster Ebene deklariert, im globalen Objekt `window`. Bei 490 Zeilen Store-Code wären das Dutzende globale Namen, die mit anderen Skripten kollidieren könnten. Die Lösung heißt **IIFE** – *Immediately Invoked Function Expression*, eine Funktion, die sofort nach ihrer Definition ausgeführt wird.

```
(function (global) {
  const STORAGE_KEY = "ftv-turnier-v1";     // privat
  function leseSpeicher() { … }             // privat
  function getTabelle()   { … }             // wird gleich veröffentlicht

  global.TurnierStore = { getTabelle, addSpiel, … };   // einzige Tür nach außen
})(window);
```

So liest man das von außen nach innen:

1. `(function (global) { … })` – eine namenlose Funktion in Klammern, damit sie als *Ausdruck* gilt.
2. `(window)` dahinter – sofort aufrufen und dabei `window` als Parameter `global` übergeben.
3. Alles, was innen mit `const` oder `function` entsteht, ist nur innerhalb dieser Funktion sichtbar.
4. Nur was ausdrücklich an `global.TurnierStore` gehängt wird, ist von außen erreichbar.

Die Kurzschreibweise `{ getTabelle, addSpiel }` ist übrigens gleichbedeutend mit `{ getTabelle: getTabelle, addSpiel: addSpiel }` – ein Objekt, dessen Schlüssel genauso heißen wie die Variablen.

> **DELPHI-VERGLEICH**
>
> Das IIFE ist JavaScripts Antwort auf den `interface`- und `implementation`-Abschnitt einer Unit. Was im `interface` steht, entspricht dem Objekt `TurnierStore`; alles im `implementation`-Teil bleibt privat. Moderne Browser kennen dafür inzwischen `<script type="module">` mit `export` – das funktioniert aber beim Öffnen über `file://` nicht, und genau deshalb ist das IIFE hier die richtige Wahl.

> **MERKE**
>
> Ein Modul hat eine kleine, öffentliche Schnittstelle und viele private Helfer. Ändert sich das

> Ein Modul hat eine kleine, öffentliche Schnittstelle und viele private Helfer. Ändert sich das Innenleben, merkt die Außenwelt nichts davon.

---

4 localStorage und JSON

`localStorage` ist ein einfacher Schlüssel-Wert-Speicher, den jeder Browser für jede Herkunft (Domain bzw. Ordner) bereitstellt. Er überlebt das Schließen des Browsers und speichert typischerweise einige Megabyte. Er hat aber eine wichtige Einschränkung: **Er kann nur Texte speichern.**

```
localStorage.setItem("zahl", 42);
localStorage.getItem("zahl");   // "42" – ein String, keine Zahl!
```

Um ganze Objekte abzulegen, wandelt man sie mit **JSON** (JavaScript Object Notation) in Text um und beim Lesen wieder zurück:

Richtung	Funktion	Ergebnis
Objekt → Text	`JSON.stringify(obj)`	`'{"version":2,"teams":[…]}'`
Text → Objekt	`JSON.parse(text)`	wieder ein echtes Objekt mit Arrays und Zahlen

Das Projekt legt den gesamten Turnierzustand unter **einem einzigen Schlüssel** ab:

```
{
  "version": 2,
  "meta":   { "turnierName": "Advends-Cup", "ort": "Stadion in Merkheim", "datum": "2026-12-0
  "teams":  [ { "id": "a1b2…", "name": "Turo Darfeld" } ],
  "spiele": [ { "id": "c3d4…", "heimId": "a1b2…", "gastId": "e5f6…", "toreHeim": 3, "toreGast
}
```

Warum ein Feld „version“?

Das Feld `version` ist eine Versicherung für die Zukunft. Kommt später ein Feld `gruppe` hinzu, kann der Store beim Lesen erkennen, dass die gespeicherten Daten noch das alte Format haben, und sie umwandeln (*Migration*). Ohne Versionsnummer müsste man das Format erraten.

Wo liegen die Daten eigentlich?

In den Entwicklerwerkzeugen des Browsers (F12) unter **Anwendung** → **Lokaler Speicher** (Chrome/Edge) bzw. **Web-Speicher** (Firefox) kann man den Schlüssel `ftv-turnier-v1` sehen und sogar bearbeiten. Das ist das beste Werkzeug zum Lernen: Ein Ergebnis eintragen, F12 öffnen und zuschauen, wie sich der JSON-Text ändert.

> **DELPHI-VERGLEICH**
>
> `localStorage` entspricht in etwa einer INI-Datei mit genau einem Eintrag oder einem `TStringList.SaveToFile`. JSON gibt es in Delphi über `System.JSON` (`TJSONObject.ParseJSONValue`) – und in TMS Web Core sogar direkt mit `TJSJSON` und `window.localStorage`, weil dort derselbe Browser darunterliegt.

> **MERKE**
>
> `localStorage` speichert nur Strings. Objekte gehen hinein mit `JSON.stringify` und kommen heraus mit `JSON.parse` – und `JSON.parse` kann bei kaputtem Text eine Ausnahme werfen.

TEIL 2

Datenlogik – die Tabelle, die nicht falsch sein kann

5 Abgeleitete Daten nicht speichern

Das ist die wichtigste Idee des ganzen Projekts, und sie gilt weit über Fussball hinaus. Man unterscheidet zwei Arten von Daten:

Art	Beispiel	Woher kommt sie?
Primärdaten	Mannschaften, Ergebnisse (3 : 1)	Ein Mensch gibt sie ein. Man kann sie nicht ausrechnen.
Abgeleitete Daten	Punkte, Siege, Tordifferenz, Platz	Lassen sich jederzeit aus den Primärdaten berechnen.

Speichert man beide, entsteht **Redundanz**: dieselbe Information steht an zwei Stellen. Wird ein Ergebnis korrigiert und die Tabelle nicht nachgezogen, widersprechen sich die beiden – und niemand weiß mehr, was stimmt. Datenbankleute nennen das eine *Update-Anomalie*.

Das Projekt löst das Problem radikal: Die Tabelle wird **bei jedem Lesen neu berechnet**. Selbst wenn in der Datei noch alte Punktzahlen stehen, werden sie überschrieben, bevor irgendwer sie sieht.

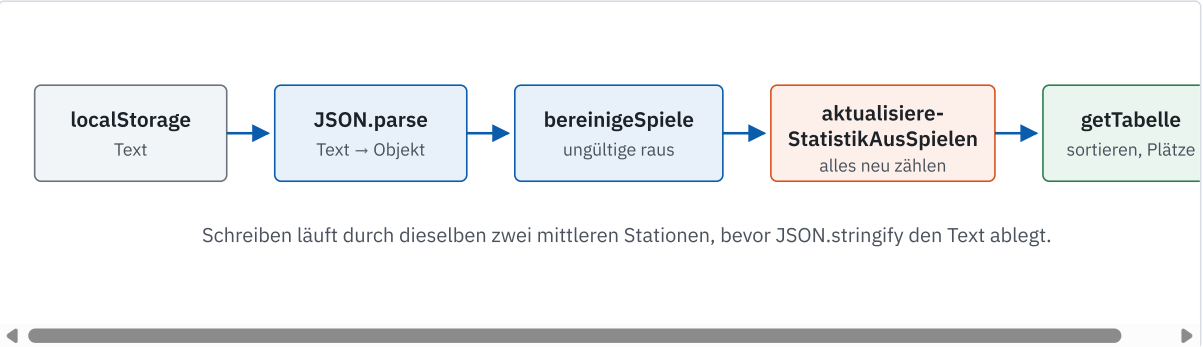


Abb. 1 – Die Lesekette. Gespeicherte Punktzahlen kommen zwar aus dem Speicher, werden aber in der orangen Station verworfen und neu berechnet.

Ist das nicht langsam?

Diese Frage stellt man sich zu Recht. Die Rechnung: Bei 20 Mannschaften und 100 Spielen werden 100 Schleifendurchläufe mit je ein paar Additionen ausgeführt. Ein heutiger Rechner schafft hunderte Millionen solcher Operationen pro Sekunde. Die Neuberechnung dauert also weit unter einer Millisekunde. **Man tauscht ein bisschen Rechenzeit gegen**

Korrektheit – bei dieser Datenmenge ein ausgezeichnetes Geschäft.

Erst bei sehr großen Datenmengen (Millionen Datensätze) würde man Ergebnisse zwischenspeichern („cachen“) – und hätte dann genau das Problem, sie aktuell halten zu müssen.

> MERKE
>
> Was man ausrechnen kann, rechnet man aus. Speichere nur, was ein Mensch eingegeben hat. Dieses Prinzip heißt auch *Single Source of Truth* – es gibt genau eine maßgebliche Quelle.

---

6 Die Tabelle Schritt für Schritt

Hier ist die Kernfunktion noch einmal, diesmal Zeile für Zeile erklärt:

```
function aktualisiereStatistikAusSpielen(teams, spiele) {
  // ① Nachschlagetabelle: id → frisch genulltes Team
  const byId = new Map(
    teams.map((t) => [t.id, { ...t, spiele: 0, siege: 0, unentschieden: 0,
      niederlagen: 0, toreErzielt: 0, toreErhalten: 0, punkte: 0 }])
  );
  // ② Jedes Spiel einmal durchgehen
  for (const s of spiele) {
    const heim = byId.get(s.heimId);
    const gast = byId.get(s.gastId);
    if (!heim || !gast) continue;          // ③ unbekannte Mannschaft → überspringen
    const th = s.toreHeim, tg = s.toreGast;
    heim.spiele++;  gast.spiele++;
    heim.toreErzielt += th; heim.toreErhalten += tg;
    gast.toreErzielt += tg; gast.toreErhalten += th;
    // ④ Punkte nach 3-1-0-Regel
    if      (th > tg) { heim.siege++; heim.punkte += 3; gast.niederlagen++; }
    else if (th < tg) { gast.siege++; gast.punkte += 3; heim.niederlagen++; }
    else   { heim.unentschieden++; gast.unentschieden++; heim.punkte++; gast.punkte++; }
  }
  // ⑤ Ursprüngliche Reihenfolge behalten
  return teams.map((t) => byId.get(t.id) || t);
}
```

① Map und Spread-Operator

`teams.map(t => [t.id, …])` erzeugt eine Liste von Paaren `[Schlüssel, Wert]`. Aus genau so einer Liste baut `new Map(…)` eine Nachschlagetabelle. Damit findet `byId.get(id)` ein Team sofort, statt jedes Mal die ganze Liste zu durchsuchen.

Der **Spread-Operator** `{ ...t, punkte: 0 }` kopiert alle Felder von `t` (Name, ID, …) in ein **neues** Objekt und überschreibt danach die genannten Felder. Die Reihenfolge ist wichtig: Was rechts steht, gewinnt. So werden alte Punkte garantiert durch 0 ersetzt, und das Original-Objekt `t` bleibt unverändert.

Ein Durchlauf von Hand

Nimm die fünf Ergebnisse des Advends-Cups aus dem Aushang. Die Tabelle zeigt, wie sich die Werte nach jedem Spiel entwickeln (Punkte / Tore):

Spiel	Turo D.	Bor. D.	SW H.	VfL B.	GW N.	Adler B.
Start	0 / 0:0	0 / 0:0	0 / 0:0	0 / 0:0	0 / 0:0	0 / 0:0
VfL Billerbeck 1:4 SW Havixbeck	0 / 0:0	0 / 0:0	**3** / 4:1	0 / 1:4	0 / 0:0	0 / 0:0
Bor. Darup 5:0 GW Nottuln	0 / 0:0	**3** / 5:0	3 / 4:1	0 / 1:4	0 / 0:5	0 / 0:0
Turo Darfeld 3:1 SW Havixbeck	**3** / 3:1	3 / 5:0	3 / 5:4	0 / 1:4	0 / 0:5	0 / 0:0
Turo Darfeld 2:1 Adler Buldern	**6** / 5:2	3 / 5:0	3 / 5:4	0 / 1:4	0 / 0:5	0 / 1:2
GW Nottuln 1:1 VfL Billerbeck	6 / 5:2	3 / 5:0	3 / 5:4	**1** / 2:5	**1** / 1:6	0 / 1:2

Die letzte Zeile stimmt exakt mit dem gedruckten Aushang überein. Probiere es selbst aus:

Labor: Tabelle live berechnen

Eine Zeile pro Spiel im Format `Heim 3:1 Gast`. Ändere ein Ergebnis oder füge eins hinzu – die Tabelle wird mit derselben Logik wie im Store neu berechnet.

Ergebnisse

```
VfL Billerbeck 1:4 SW Havixbeck
Bor. Darup 5:0 GW Nottuln
Turo Darfeld 3:1 SW Havixbeck
Turo Darfeld 2:1 Adler Buldern
GW Nottuln 1:1 VfL Billerbeck
```

Pl.	Mannschaft	Sp	S	U	N	Tore	Diff	Pkt
1	Turo Darfeld	2	2	0	0	5 : 2	+3	**6**
2	Bor. Darup	1	1	0	0	5 : 0	+5	**3**
3	SW Havixbeck	2	1	0	1	5 : 4	+1	**3**
4	VfL Billerbeck	2	0	1	1	2 : 5	-3	**1**
5	GW Nottuln	2	0	1	1	1 : 6	-5	**1**
6	Adler Buldern	1	0	0	1	1 : 2	-1	**0**

> **MERKE**
>
> `Map` für schnelles Nachschlagen per Schlüssel, `{ ...obj, feld: wert }` für eine veränderte Kopie ohne das Original anzufassen. Beides zusammen ist das tägliche Brot moderner JavaScript-Programmierung.

7 Sortieren mit Vergleichsfunktion

`Array.prototype.sort` bekommt eine **Vergleichsfunktion**, die für zwei Elemente sagt, welches zuerst kommt. Das Vorzeichen des Rückgabewerts entscheidet:

Rückgabe	Bedeutung
< 0	`links` kommt vor `rechts`
> 0	`rechts` kommt vor `links`
0	gleichwertig, Reihenfolge bleibt

```
teams.sort((links, rechts) => {
  if (links.punkte !== rechts.punkte)             return rechts.punkte - links.punkte;
  if (links.tordifferenz !== rechts.tordifferenz) return rechts.tordifferenz - links.tordiffe
  if (links.toreErzielt !== rechts.toreErzielt)   return rechts.toreErzielt - links.toreErzie
```

```
  return links.name.localeCompare(rechts.name, "de");
});
```

Der Trick `rechts.punkte - links.punkte` sortiert **absteigend**: Hat rechts mehr Punkte, ist die Differenz positiv, also wandert rechts nach vorn. Für aufsteigend dreht man die Subtraktion um.

Die Kaskade aus `if`-Abfragen bildet eine **mehrstufige Rangfolge**: Erst wenn die Punkte gleich sind, wird die Tordifferenz befragt, und so weiter. Im Beispiel haben Bor. Darup und SW Havixbeck je 3 Punkte; die Tordifferenz +5 gegen +1 entscheidet für Darup.

Warum der Name als letztes Kriterium?

Ohne Namen wären zwei Teams mit völlig gleichen Werten „gleichwertig“. JavaScript sortiert seit ES2019 garantiert **stabil** – die ursprüngliche Reihenfolge bliebe also erhalten. Die hängt aber davon ab, wann ein Team angelegt wurde, und wirkt für Zuschauer zufällig. Der Name macht das Ergebnis vorhersagbar. `localeCompare(…, "de")` sortiert dabei Umlaute korrekt: „Ölde“ landet bei O, nicht hinter Z.

> **DELPHI-VERGLEICH**
>
> In Delphi übernimmt `TComparer<T>.Construct` die Rolle der Vergleichsfunktion (siehe Kapitel 16). Achtung: `TArray.Sort` arbeitet mit Quicksort und ist **nicht stabil** – dort ist ein eindeutiges letztes Kriterium sogar Pflicht, sonst springen gleichwertige Teams bei jedem Sortieren hin und her.

---

8 Defensive Programmierung

Daten im Browser sind nicht sicher vor Veränderung: Jemand bearbeitet sie in den Entwicklerwerkzeugen, eine fremde Importdatei ist fehlerhaft, oder eine gelöschte Mannschaft hinterlässt verwaiste Spiele. Der Store geht deshalb davon aus, dass **alles kaputt sein könnte**, und schützt sich auf drei Ebenen.

Ebene 1: Kein Absturz

```
try {
  const data = JSON.parse(raw);
  if (!data || !Array.isArray(data.teams)) return leererZustand();
  …
} catch {
  return leererZustand();     // lieber leer starten als gar nicht
}
```

Das `catch` ohne Variable (`catch {` statt `catch (e) {`) ist moderne Syntax für „Fehler egal, nur abfangen“. Abgefangen werden kaputtes JSON, ein abgeschalteter Speicher und der private Modus mancher Browser.

Ebene 2: Selbstheilung

`bereinigeSpiele()` filtert bei **jedem** Lesen und Schreiben alle Spiele heraus, deren Mannschaften fehlen, die gegen sich selbst gespielt wurden oder deren Tore keine Zahlen

sind. Kaputte Daten verschwinden dadurch von allein – man muss keine Reparaturfunktion aufrufen.

Ebene 3: Statusobjekte statt Ausnahmen

Schreibende Funktionen geben ein einheitliches Objekt zurück:

```
return { ok: false, error: "Diese Mannschaft existiert bereits." };
return { ok: true,  teams: getMannschaften() };
```

Dieses Muster hat zwei Vorteile gegenüber `throw`: Der Aufrufer kann den Fehler nicht versehentlich übersehen (er muss `result.ok` prüfen, um an die Daten zu kommen), und die **Fehlermeldung steht an einer einzigen Stelle**. Alle sieben Seiten sagen bei demselben Problem denselben Satz.

> **DELPHI-VERGLEICH**
>
> In Delphi entspricht das einer Funktion, die `Boolean` zurückgibt und den Fehlertext über einen `out`-Parameter liefert – wie `TryStrToInt(S, Wert)`. Ausnahmen (`raise`) sind für Unerwartetes gedacht; dass ein Name doppelt eingegeben wird, ist dagegen ein ganz normaler Fall.

> **MERKE**
>
> Vertraue keinen Daten, die von außen kommen – auch nicht den eigenen von gestern. Prüfe beim Lesen, nicht nur beim Schreiben.

---

9 IDs statt Namen

Ein Spiel speichert nicht „Turo Darfeld gegen SW Havixbeck“, sondern `heimId` und `gastId`. Der Grund: Namen ändern sich. Wird aus „SW Havixbeck“ später „SuS Havixbeck“, bleiben alle Ergebnisse erhalten, weil sie an der unveränderlichen ID hängen. Das ist dasselbe Prinzip wie ein Primärschlüssel in einer Datenbank.

```
function neueId() {
  if (global.crypto && crypto.randomUUID) return crypto.randomUUID();
  return "id-" + Date.now() + "-" + Math.random().toString(36).slice(2, 9);
}
```

`crypto.randomUUID()` liefert eine weltweit eindeutige Kennung wie `3f2a…-4b1c-…`. Sie ist aber nur in einem **sicheren Kontext** verfügbar – also über `https://` oder `localhost`. Beim Öffnen per Doppelklick (`file://`) fehlt sie in manchen Browsern. Darum der Rückfallweg:

- `Date.now()` – Millisekunden seit 1970, ändert sich ständig,
- `Math.random().toString(36)` – Zufallszahl im 36er-System (Ziffern und Buchstaben),
- `.slice(2, 9)` – schneidet das führende `0.` ab und nimmt sieben Zeichen.

Diese Technik heißt *Feature Detection*: Man prüft, ob eine Funktion existiert, statt den Browser anhand seines Namens zu erraten.

> **DELPHI-VERGLEICH**
>
> In Delphi liefert `TGUID.NewGuid.ToString` (Unit `System.SysUtils`) dasselbe – ohne Rückfallweg, weil das Betriebssystem es immer kann.

TEIL 3

Oberfläche – vom Ereignis zum Bild

10 DOM, Events und Sicherheit

Das **DOM** (Document Object Model) ist der Baum aus Objekten, den der Browser aus dem HTML baut. JavaScript liest und verändert diesen Baum – genau wie Delphi-Code die Komponenten eines Formulars anspricht.

Aufgabe	JavaScript	Delphi (VCL)
Element finden	`document.querySelector("#feldName")`	`EditName` (Komponente im Formular)
Text setzen	`el.textContent = "Tabelle"`	`Label1.Caption := 'Tabelle'`
Auf Klick reagieren	`btn.addEventListener("click", fn)`	`Button1.OnClick := Button1Click`
Ausblenden	`el.hidden = true`	`Panel1.Visible := False`

Ein Unterschied: `addEventListener` kann **mehrere** Behandler für dasselbe Ereignis anhängen; in Delphi gibt es pro Ereignis genau einen.

innerHTML und textContent

Beide setzen Inhalt, aber grundverschieden:

```
zelle.innerHTML   = team.name;  // Browser liest den Text als HTML!
zelle.textContent = team.name;  // Browser zeigt den Text wörtlich
```

Heißt eine Mannschaft `<img src=x onerror=alert(1)>`, führt `innerHTML` den Code aus. Diese Angriffsart heißt **Cross-Site-Scripting (XSS)**. Bei einer rein lokalen App ist das Risiko gering – aber eine Importdatei könnte von jemand anderem stammen. Das Projekt geht deshalb zwei Wege: Ältere Dateien schicken Namen durch `escapeHtml()`, neuere benutzen durchgehend `textContent`.

```
function escapeHtml(text) {
  return String(text)
    .replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;").replace(/'/g, "&#39;");
}
```

Wichtig ist die Reihenfolge: `&` muss zuerst ersetzt werden, sonst würde aus dem bereits erzeugten `<` ein `<`.

data-Attribute als Bindeglied

`datentransfer.js` sucht seine Knöpfe nicht über IDs, sondern über Attribute wie `data-aktion="export"`. Damit kann jede Seite die Sichern-Funktion bekommen, indem sie nur das Markup ergänzt – das Skript bleibt unverändert. Man nennt das *lose Kopplung*.

> **MERKE**
>
> Benutzereingaben gehören mit `textContent` ins DOM. `innerHTML` nur für eigenes, festes Markup – oder mit vorherigem Escapen.

11 Seiten synchron halten

Das Projekt kann die Tabelle auf einem zweiten Bildschirm zeigen, während im ersten Fenster Ergebnisse eingegeben werden. Möglich machen das zwei Ereignisse, die sich ergänzen:

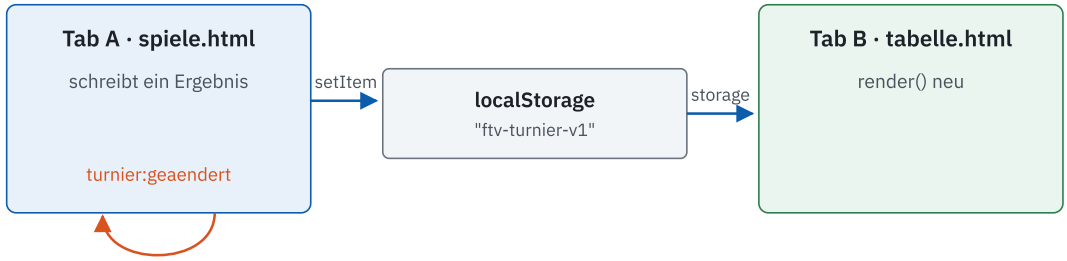


Abb. 2 – Zwei Wege der Benachrichtigung. Blau: vom Browser geliefert. Orange: selbst ausgelöst.

```
// Empfänger – auf jeder Seite
window.addEventListener("turnier:geaendert", render);
window.addEventListener("storage", (e) => {
  if (e.key === "ftv-turnier-v1") render();
});

// Sender – z. B. nach einem Import in datentransfer.js
window.dispatchEvent(new CustomEvent("turnier:geaendert"));
```

Der Doppelpunkt im Namen `turnier:geaendert` hat keine technische Bedeutung. Er ist eine Konvention, um eigene Ereignisse klar von eingebauten wie `click` zu unterscheiden.

> **DELPHI-VERGLEICH**
>
> Ein `CustomEvent` ist wie ein selbst definiertes Ereignis `OnTurnierGeaendert: TNotifyEvent`, das eine Klasse auslöst. Das `storage`-Ereignis zwischen Tabs ähnelt eher einer Windows-Nachricht (`PostMessage`) zwischen zwei Prozessen.

12 Tastatur: `key` und `code`

Ein Tastendruck liefert zwei verschiedene Angaben, und die Live-Anzeige ist genau an

diesem Unterschied gescheitert:

Eigenschaft	Bedeutet	Shift + 1 (deutsch)	Taste Z (deutsch)
`event.key`	das erzeugte **Zeichen**	`!`	`z`
`event.code`	die **physische Taste** (US-Lage)	`Digit1`	`KeyY`

Das Beispiel rechts zeigt die Falle bei `code`: Er benennt die Taste nach ihrer Position auf einer amerikanischen Tastatur, deshalb ist das deutsche Z dort `KeyY`. Für Ziffern und die im Projekt benutzten Tasten H, F und N spielt das keine Rolle.

Labor: Was meldet deine Tastatur?

Klicke in das Feld und drücke Tasten – auch mit Shift. Besonders lehrreich: Shift + 1 und die Taste Z.

hier klicken, dann Taste drücken

EVENT.KEY	EVENT.CODE	SHIFTKEY	AKTION IM PROJEKT
–	–	–	–

```
const delta = e.shiftKey ? -1 : 1;
switch (e.code) {
  case "Digit1": case "Numpad1": aendereTore("heim", delta); break;
  case "Digit2": case "Numpad2": aendereTore("gast", delta); break;
}
```

Die Lösung trennt zwei Fragen sauber: *Welche Taste?* beantwortet `e.code`, *mit oder ohne Shift?* beantwortet `e.shiftKey`. Vorher musste das Zeichen `!` beide Fragen gleichzeitig beantworten – und das ist vom Tastaturlayout abhängig.

MERKE

`event.key` für Texteingaben, `event.code` plus `shiftKey` für Tastenkürzel.

13 CSS-Werkzeuge

Variablen (Custom Properties)

```
:root { --color-orange: #e85a2a; --radius: 8px; }
.knopf { background: var(--color-orange); border-radius: var(--radius); }
```

`:root` ist das oberste Element der Seite. Was dort definiert wird, ist überall verfügbar. Ein einziger geänderter Wert färbt die gesamte Anwendung um – wie eine Konstante in einer Delphi-Unit, die von allen Formularen benutzt wird.

Grid für die Live-Anzeige

```
.live-anzeige { display: grid; grid-template-rows: auto 1fr auto; }
```

Drei Zeilen: `auto` (so hoch wie die Uhr), `1fr` (der gesamte Rest), `auto` (so hoch wie die Fußzeile). Die Einheit `fr` bedeutet „Anteil am freien Platz“. Dadurch sitzt der Spielstand immer in der Mitte, egal wie hoch das Fenster ist – vergleichbar mit `Align := alTop`, `alClient`, `alBottom` in Delphi.

clamp() für mitwachsende Schrift

```
.live-stand { font-size: clamp(3.5rem, 15vw, 15rem); }
```

Fensterbreite	15vw	Ergebnis	Begründung
375 px (Handy)	56 px	56 px	gleich dem Minimum 3,5 rem × 16 px
1280 px (Beamer)	192 px	192 px	liegt zwischen den Grenzen
1920 px (Full HD)	288 px	240 px	gedeckelt bei 15 rem × 16 px

Medienabfragen

```
.th-kurz { display: none; }
@media (max-width: 820px) {
  .th-lang { display: none; }
  .th-kurz { display: inline; }
}
```

Beide Beschriftungen stehen im HTML, CSS entscheidet nach Fensterbreite. Bemerkenswert ist die Grenze 820 px: Sie wurde nicht geraten, sondern aus dem Platzbedarf der Spalten berechnet (Kapitel 6 der Projekt-Doku). Gute Umschaltpunkte richten sich nach dem Inhalt, nicht nach Gerätetypen.

Druck mit @media print

```
@media print {
  @page { size: A4 portrait; margin: 14mm; }
  .no-print { display: none !important; }
  .aushang-tabelle tbody tr { break-inside: avoid; }
  .aushang-tabelle thead { display: table-header-group; }
}
```

Dieselbe HTML-Seite hat zwei Erscheinungsbilder: eines für den Bildschirm, eines für das Papier. Für Delphi-Entwickler ist das, als hätte jedes Formular automatisch einen eingebauten Report.

Die hidden-Falle

Das HTML-Attribut `hidden` blendet ein Element aus – es sei denn, eine eigene CSS-Regel setzt `display: flex`. Dann gewinnt die Regel, und das Element bleibt sichtbar. Die Abhilfe aus dem Projekt: `.klasse[hidden] { display: none; }`. Hier zeigt sich ein Grundprinzip von CSS: Bei Konflikten entscheidet die **Spezifität** der Selektoren.

---

14 Datum und Zeitzonen

Ein Datumsfeld liefert Text wie `2026-12-05`. Wandelt man das mit `new Date("2026-12-05")` um, nimmt JavaScript **Mitternacht UTC** an. In Deutschland ist das 1 Uhr morgens – kein Problem. In New York ist es aber 19 Uhr am **Vortag**, und die Anzeige stünde dort auf Freitag statt Samstag.

```
const d = new Date(meta.datum + "T12:00:00");   // Mittag lokale Zeit
new Intl.DateTimeFormat("de-DE", {
  weekday: "long", day: "numeric", month: "long", year: "numeric"
}).format(d);   // "Samstag, 5. Dezember 2026"
```

Mit Uhrzeitangabe, aber **ohne** Zeitzone wird der Text als Ortszeit gelesen. Mittag ist der sicherste Punkt: Von dort aus sind es zu jeder Zeitzone der Erde weniger als 12 Stunden bis zum Datumswechsel.

> **DELPHI-VERGLEICH**
>
> `TDateTime` in Delphi kennt keine Zeitzone, deshalb tritt das Problem dort selten auf. Die Entsprechung zu `Intl.DateTimeFormat` ist `FormatDateTime('dddd, d. mmmm yyyy', Datum)` mit deutschen `FormatSettings`.

> **MERKE**
>
> Ein reines Datum ist kein Zeitpunkt. Wer ein Datum ohne Uhrzeit in ein `Date` verwandelt, hängt `T12:00:00` an.

---

15 Sichern und Laden: Blob, FileReader, Promise

`localStorage` gehört einem Browser auf einem Rechner. Die Exportdatei ist deshalb die einzige Brücke zu einem anderen Gerät und gleichzeitig die Datensicherung.

Export: eine Datei ohne Server

1. `new Blob([text], { type: "application/json" })` – ein Datenpaket im Arbeitsspeicher.
2. `URL.createObjectURL(blob)` – eine temporäre Adresse dafür (`blob:…`).
3. Ein unsichtbarer Link mit `download`-Attribut wird erzeugt und per Code angeklickt.
4. `URL.revokeObjectURL(url)` nach einer Sekunde – gibt den Speicher wieder frei.

Import: asynchron lesen

Das Lesen einer Datei dauert, und der Browser darf dabei nicht einfrieren. Der `FileReader` arbeitet deshalb **asynchron**: Man startet das Lesen, und später ruft der Browser `onload` auf. Das Projekt verpackt das in ein **Promise**:

```
function importStateAusDatei(file) {
  return new Promise((resolve) => {
    const reader = new FileReader();
    reader.onload  = () => resolve(importState(JSON.parse(String(reader.result))));
    reader.onerror = () => resolve({ ok: false, error: "Die Datei konnte nicht gelesen werden
    reader.readAsText(file, "utf-8");
  });
```

```
}

// Aufrufer – liest sich wie normaler, sequenzieller Code
const result = await TurnierStore.importStateAusDatei(datei);
if (!result.ok) showMessage(result.error, "error");
```

Ein Promise ist ein Versprechen auf ein späteres Ergebnis. `await` wartet darauf, ohne den Browser zu blockieren. Auffällig: Das Projekt ruft nie `reject` auf, sondern meldet auch Fehler über `resolve({ ok: false })`. Damit bleibt es beim einheitlichen Statusobjekt aus Kapitel 8.

> **MERKE**
>
> Daten aus einer Importdatei sind fremde Daten. `importState()` prüft Kennung, Namen, IDs und Torzahlen einzeln – erst dann landet etwas im Speicher.

TEIL 4

Vertiefen – übertragen, üben, festigen

16 Dieselbe Logik in Delphi

Die Web-Version ist die Schwester der Delphi-Anwendung FussballTurnier 2.0. Die folgende Unit überträgt die Kernideen aus den Kapiteln 5 bis 7 eins zu eins: Statistik aus den Spielen neu zählen, mehrstufig sortieren, Plätze vergeben. Sie läuft in der VCL, in FMX und mit kleinen Anpassungen auch in TMS Web Core.

uTurnierTabelle.pas

```
unit uTurnierTabelle;

interface

uses
  System.SysUtils, System.Generics.Collections, System.Generics.Defaults;

type
  TSpiel = record
    HeimId, GastId: string;
    ToreHeim, ToreGast: Integer;
  end;

  TTeamStat = record
    Id, Name: string;
    Spiele, Siege, Unentschieden, Niederlagen: Integer;
    ToreErzielt, ToreErhalten, Punkte, Platz: Integer;
    function Tordifferenz: Integer;
    procedure Nullen;
  end;

{ Berechnet die Tabelle ausschließlich aus Teams und Spielen.
  Gespeicherte Statistikwerte in Teams werden ignoriert. }
function BerechneTabelle(const Teams: TArray<TTeamStat>;
  const Spiele: TArray<TSpiel>): TArray<TTeamStat>;
```

```
implementation

function TTeamStat.Tordifferenz: Integer;
begin
  Result := ToreErzielt - ToreErhalten;
end;

procedure TTeamStat.Nullen;
begin
  Spiele := 0; Siege := 0; Unentschieden := 0; Niederlagen := 0;
  ToreErzielt := 0; ToreErhalten := 0; Punkte := 0; Platz := 0;
end;

function BerechneTabelle(const Teams: TArray<TTeamStat>;
  const Spiele: TArray<TSpiel>): TArray<TTeamStat>;
var
  Index: TDictionary<string, Integer>;   // entspricht der JS-Map byId
  I, H, G: Integer;
  S: TSpiel;
begin
  Result := Copy(Teams);                      // Kopie – Original bleibt unberührt
  Index := TDictionary<string, Integer>.Create;
  try
    for I := 0 to High(Result) do
    begin
      Result[I].Nullen;                       // wie { ...t, punkte: 0 }
      Index.AddOrSetValue(Result[I].Id, I);
    end;

    for S in Spiele do
    begin
      // bereinigen: unbekannte Teams und Spiel gegen sich selbst überspringen
      if not Index.TryGetValue(S.HeimId, H) then Continue;
      if not Index.TryGetValue(S.GastId, G) then Continue;
      if H = G then Continue;
      if (S.ToreHeim < 0) or (S.ToreGast < 0) then Continue;

      Inc(Result[H].Spiele);  Inc(Result[G].Spiele);
      Inc(Result[H].ToreErzielt, S.ToreHeim); Inc(Result[H].ToreErhalten, S.ToreGast);
      Inc(Result[G].ToreErzielt, S.ToreGast); Inc(Result[G].ToreErhalten, S.ToreHeim);

      if S.ToreHeim > S.ToreGast then
      begin
        Inc(Result[H].Siege); Inc(Result[H].Punkte, 3); Inc(Result[G].Niederlagen);
      end
      else if S.ToreHeim < S.ToreGast then
      begin
        Inc(Result[G].Siege); Inc(Result[G].Punkte, 3); Inc(Result[H].Niederlagen);
      end
      else
      begin
        Inc(Result[H].Unentschieden); Inc(Result[G].Unentschieden);
        Inc(Result[H].Punkte); Inc(Result[G].Punkte);
      end;
    end;
  finally
    Index.Free;
  end;

  // Rangfolge: Punkte → Tordifferenz → Tore → Name (Quicksort ist nicht stabil!)
  TArray.Sort<TTeamStat>(Result, TComparer<TTeamStat>.Construct(
    function(const L, R: TTeamStat): Integer
    begin
```

```
      Result := R.Punkte - L.Punkte;
      if Result = 0 then Result := R.Tordifferenz - L.Tordifferenz;
      if Result = 0 then Result := R.ToreErzielt - L.ToreErzielt;
      if Result = 0 then Result := AnsiCompareText(L.Name, R.Name);
    end));

  for I := 0 to High(Result) do
    Result[I].Platz := I + 1;
end;

end.
```

Gegenüberstellung

JavaScript	Delphi	Hinweis
`new Map()`	`TDictionary<string, Integer>`	Delphi muss freigegeben werden (`try…finally`)
`{ ...t, punkte: 0 }`	`Copy(Teams)` + `Nullen`	Records werden in Delphi ohnehin als Wert kopiert
`byId.get(id)` + `if (!heim)`	`TryGetValue(id, H)`	Delphi liefert Erfolg und Wert in einem Aufruf
`teams.sort((l, r) => …)`	`TArray.Sort` + `TComparer.Construct`	anonyme Methode statt Pfeilfunktion
`localeCompare(…, "de")`	`AnsiCompareText`	beide berücksichtigen Umlaute
stabile Sortierung (ES2019)	Quicksort, nicht stabil	eindeutiges letztes Kriterium nötig

Aufgerufen wird die Unit etwa so, um ein `TStringGrid` zu füllen:

```
Tabelle := BerechneTabelle(FTeams, FSpiele);
for I := 0 to High(Tabelle) do
  with Tabelle[I] do
  begin
    Grid.Cells[0, I + 1] := IntToStr(Platz);
    Grid.Cells[1, I + 1] := Name;
    Grid.Cells[2, I + 1] := Format('%d : %d', [ToreErzielt, ToreErhalten]);
    Grid.Cells[3, I + 1] := IntToStr(Punkte);
  end;
```

17 Übungen

Die Übungen bauen auf dem echten Projekt auf und greifen teilweise die offenen Punkte aus der Projekt-Dokumentation auf. Erst selbst versuchen, dann aufklappen.

Ü1 Im Aushang hat GW Nottuln 1 Punkt und steht hinter VfL Billerbeck mit ebenfalls 1 Punkt. Welches Kriterium entscheidet, und mit welchen Werten?

▸ **Lösung**

Ü2 Du öffnest F12, suchst im lokalen Speicher bei „Turo Darfeld“ das Feld `punkte` und setzt es auf 99. Was zeigt die Tabelle nach dem Neuladen – und warum?

▸ Lösung

Ü3 Die Sortier-Auswahl auf `tabelle.html` hat nur eine Option (offener Punkt der Projekt-Doku). Erweitere `getTabelle` um einen Parameter, der zusätzlich „nach Name“ sortieren kann, ohne die Platzierung zu verändern.

▸ Lösung

Ü4 `escapeHtml()` steht dreimal identisch im Projekt. Wie bringst du sie an eine gemeinsame Stelle, ohne einen Bundler einzuführen?

▸ Lösung

Ü5 Bereite das Datenmodell auf Gruppen vor: Jede Mannschaft bekommt ein Feld `gruppe` („A“ oder „B“). Wie ändert sich `getTabelle`, und was passiert mit gespeicherten Daten ohne dieses Feld?

▸ Lösung

Ü6 Warum muss in `escapeHtml` das `&` als erstes Zeichen ersetzt werden? Spiele es mit dem Text `<b>` durch.

▸ Lösung

Ü7 · Delphi Erweitere `uTurnierTabelle` um eine Funktion `LiesSpieleAusText(const Zeilen: TStrings): TArray<TSpiel>`, die Zeilen im Format `Heim 3:1 Gast` einliest – wie das Labor in Kapitel 6.

▸ Lösung

18 Glossar und Zusammenfassung

Begriff	Erklärung
DOM	Objektbaum, den der Browser aus HTML baut und den JavaScript verändert.
IIFE	Sofort ausgeführte Funktion; schafft einen privaten Bereich für ein Modul.
JSON	Textformat für Objekte, Arrays, Zahlen und Texte; `stringify` / `parse`.
localStorage	Dauerhafter Schlüssel-Wert-Speicher des Browsers, nur für Strings.
Abgeleitete Daten	Werte, die sich aus anderen berechnen lassen; werden hier nicht gespeichert.
Single Source of Truth	Jede Information hat genau eine maßgebliche Quelle.
Map	Nachschlagetabelle Schlüssel → Wert, in Delphi `TDictionary`.
Spread-Operator	`{ ...obj }` kopiert alle Felder in ein neues Objekt.
Vergleichsfunktion	Liefert <0, 0 oder >0 und steuert damit `sort`.
Stabile Sortierung	Gleichwertige Elemente behalten ihre Reihenfolge.

Statusobjekt	`{ ok, error }` statt Ausnahme für erwartbare Fehler.
UUID	Weltweit eindeutige Kennung, z. B. aus `crypto.randomUUID()`.
Feature Detection	Prüfen, ob eine Funktion existiert, bevor man sie benutzt.
XSS	Eingeschleuster Code über Text, der als HTML interpretiert wird.
CustomEvent	Selbst definiertes Ereignis, ausgelöst mit `dispatchEvent`.
Promise / await	Platzhalter für ein späteres Ergebnis; `await` wartet ohne zu blockieren.
Blob	Binäres Datenpaket im Arbeitsspeicher, z. B. eine Exportdatei.
Medienabfrage	`@media`-Regel, die nur unter Bedingungen gilt (Breite, Druck).
clamp()	Wert mit Unter- und Obergrenze: `clamp(min, wunsch, max)`.
Spezifität	Gewicht eines CSS-Selektors; entscheidet bei widersprüchlichen Regeln.

Die fünf Sätze, die man mitnehmen sollte

> **ZUSAMMENFASSUNG**
>
> - **Eine Datenschicht:** Nur ein Modul kennt den Speicher; alle anderen fragen es.
> - **Nichts Berechenbares speichern:** Dann kann es auch nicht veralten oder sich widersprechen.
> - **Allem misstrauen, was gelesen wird:** bereinigen, abfangen, mit Statusobjekten antworten.
> - **Benutzertext bleibt Text:** `textContent` statt `innerHTML`.
> - **Die Plattform kennen:** `e.code` für Kürzel, `T12:00:00` für Daten, `storage` nur in fremden Tabs.

Das Projekt zeigt, dass man für eine vollständige Anwendung kein Framework braucht – wohl aber klare Entscheidungen. Wer diese Entscheidungen versteht, kann sie auf jedes andere Projekt übertragen: auf eine Kasse, eine Smart-Home-Steuerung oder die nächste Turnierverwaltung mit Gruppen und KO-Runde.



Erstellt: 2026-10-09 · Lern-Abhandlung zur Web-Anwendung Fussball Turnier · HTML / CSS / Vanilla JavaScript · Rw